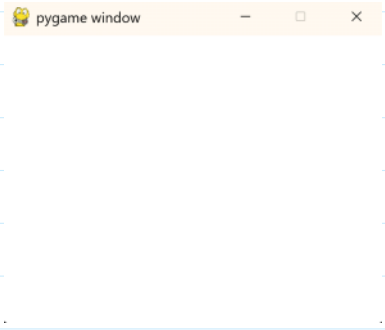
Introduction to PyGame

14 October 2025 20:09

PyGame is a Python module designed specifically for game development. It provide a set of tools and libraries for building games and multimedia applications. Pie Game was first created by Pete **Shinner in 2000**, as a side project to explore Python's multimedia capabilities. He released the first version of pie game in March 2000, which included basic functionality such as image loading, sound playback and event handling. Over the years, PyGame has grown and evolved with the Python language adding new features and capabilities. In 2007, the PyGame community created a pygame subset for Android, which allowed Pygame application to run on Android devices. Today pygame is widely used by game developers and enthusiast alike and its popularity continues to grow as Python become increasingly popular as a programming language for game development.

```
import pygame
import sys
# Initialize the game.
pygame.init()
# Create the screen.
gameWindow = pygame.display.set_mode((320,240))
WHITE = (255, 255, 255)
while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            sys.exit()
    gameWindow.fill(WHITE)
    pygame.display.flip()
```

Output:



The **pygame.init() i**nitializes pygame and the pygame.display get us to the game window. while True: is out game loop that loops through our game forever or until someone closes the pygame screen which triggers a quit event.
The **pygame.display.flip** function updates the contents of the entire display. It is typically used after drawing or updating the display to make sure that the changes are visible. The function is also known as "screen flip" or "page flip" as it swaps the front and back buffers which contains the contents of the display.

Hello World

To add a Hello world text to our blank screen, we need to create a font object. So the font = pygame.font.Font(None, 32) line creates a font object with size 32. This font object can then be used to render text onto a display Surface. The nun argument specifies that the default font should be used. If a font file is passed in, then that

font will be used instead. The font size is specified in points where. One point is 1/72 of an inch.
The font.render() function is used to render text onto a display surface. It takes three arguments: The text to be rendered. Whether the text should be anti-aliased, and the color of the text. In this case, the font.render() function is used to render the text "Hello World" with anti-aliasing turned on and the color is black. The rendered text is then stored in the text surface.

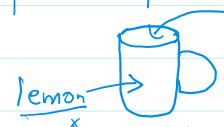
To put the text on the screen, we first need to split the text object onto the screen surface. We will use the blit function to draw the text surface onto the screen at the center. The screen.blit() Function takes two arguments: The surface to be drawn and the position at which it should be drawn. The first argument in this case is the text surface that was created on the previous font.render. The second argument is a tuple containing the coordinates of the center of the screen. ***Which is calculated by subtracting half of the width and height of the text surface from the width and height of the screen.*** This allows the text to be centered on the screen.

256^3 = 16,777,216

1.6 billion colours can be produced

	Red	Green	Blue
WHITE	(255	255	255)
Black	(0	0	0)
Red	(255	0	0)
Green	(0	255	0)
Blue	(0	0	255)

→ Boil ginger (5 mins)
put a pinch of tea (any)

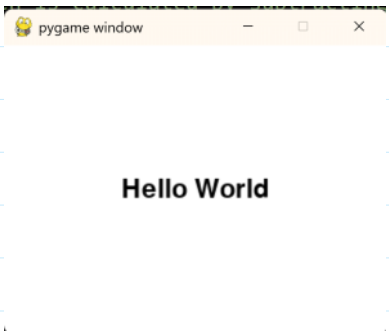


lemon → try to remove the colour of caffeine

```
import pygame
import sys
# Initialize the game.
pygame.init()
# Create the screen.
screen = pygame.display.set_mode((320, 240))
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
# Create the font object.
font = pygame.font.Font(None, 32)
while True:
    screen.fill(WHITE) # Fill the background.

    # Check for quit event.
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            sys.exit()
```

```
    # Create the text surface.
    text = font.render('Hello World', True, BLACK)

    # blit the text surface to the center of the screen.
    # Which is calculated by subtracting half of the width and
    # height of the text surface from the width and height of the screen.
    screen.blit(text, ((screen.get_width()-text.get_width())/2,
                       (screen.get_height() - text.get_height())/2))
    #update the screen
    pygame.display.flip()
```

Output:



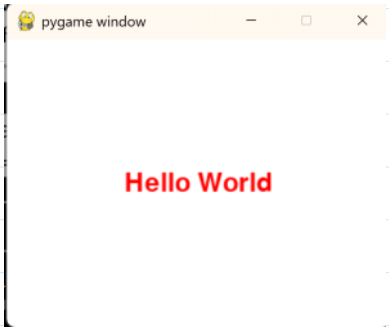
Blinking Hello World

Let's say we want to blink Hello World black and red every second. We could add a time delay in our game loop and alternate different color text object. Here we take advantage of a Python list to alternate between colors when creating the text object.

```
import pygame
import sys
import random
# Initialize the game.
pygame.init()
# Create the screen.
screen = pygame.display.set_mode((320, 240))
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (255, 0, 0)
GREEN = (0,255,0)
BLUE = (0, 0, 255)
HelloWorldColors = [BLACK, RED, GREEN, BLUE]
# Create the font object.
font = pygame.font.Font(None, 32)
# count = 0
while True:
    # count += 1
    screen.fill(WHITE) # Fill the background.

    # Check for quit event.
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            sys.exit()
```

```
    # Create the text surface.
    text = font.render('Hello World', True, HelloWorldColors[random.randint(0,3)])

    # Wait one second (or 1000 milliseconds)
    pygame.time.delay(1000)
    # blit the text surface to the center of the screen.
    # Which is calculated by subtracting half of the width and
    # height of the text surface from the width and height of the screen.
    screen.blit(text, ((screen.get_width()-text.get_width())/2,
                       (screen.get_height() - text.get_height())/2))
    #update the screen
    pygame.display.flip()
```

Output:



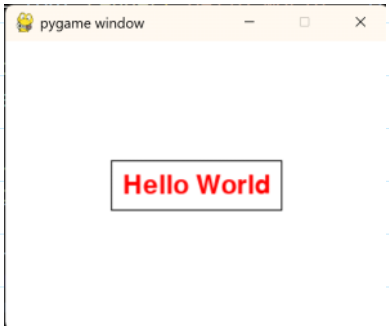
Surrounding with a border

What if we wanted to place a black border around the blinking Hello World?
pygame.draw.rect() function take 5 arguments: the display surface to draw on, the color of the rectangle, the width and height of the rectangle, and the thickness of the line. In this case, the top left corner of the rectangle is calculated by subtracting 10 from the coordinates of the center of the screen, and the width and height of the rectangle is calculated by adding 20 to the width and height of the text surface. The one argument specifies that the line should be one pixel thick.

```
import pygame
import sys
import random
# Initialize the game.
pygame.init()
# Create the screen.
screen = pygame.display.set_mode((320, 240))
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (255, 0, 0)
GREEN = (0,255,0)
BLUE = (0, 0, 255)
HelloWorldColors = [BLACK, RED, GREEN, BLUE]
# Create the font object.
font = pygame.font.Font(None, 32)
# count = 0
while True:
    # count += 1
    screen.fill(WHITE) # Fill the background.
```

```
    # Check for quit event.
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            sys.exit()

    # Create the text surface.
    text = font.render('Hello World', True, HelloWorldColors[random.randint(0,3)])

    # Wait one second (or 1000 milliseconds)
    pygame.time.delay(1000)

    # # Surround text with black rectangular border.
    # In this case, the top left corner of the rectangle is calculated by subtracting 10
    # from the coordinates of the center of the screen, and the width and height of the
    # rectangle is calculated by adding 20 to the width and height of the text surface.
    # The one argument specifies that the line should be one pixel thick.
    pygame.draw.rect(screen, BLACK, ((screen.get_width() - text.get_width())/2 -10,
                     (screen.get_height() - text.get_height())/2 - 10, text.get_width()
+ 20,
                     text.get_height() + 20), 1)

    # blit the text surface to the center of the screen.
    # Which is calculated by subtracting half of the width and
    # height of the text surface from the width and height of the screen.
    screen.blit(text, ((screen.get_width()-text.get_width())/2,
                       (screen.get_height() - text.get_height())/2))

    #update the screen
    pygame.display.flip()
```

Output:



Home Wor

1. Game Window and Background Setup

Write a Python program using PyGame to:

- Initialize a game window of size 800×600.
- Fill the background with a custom color (light blue).
- Continuously display the window until the user closes it.
- Print “Window Closed” in the console when the game is exited.

2. Display “Welcome to PyGame” Centered on Screen

Create a PyGame program that:

- Displays the text “Welcome to PyGame” at the center of a white screen.
- Uses a custom font size of 48.

3. Changing Text Colors Automatically

Build a PyGame application that:

- Displays “Hello World” in the center of the screen.
- Cycles through 5 different colors (Red, Green, Blue, Yellow, Purple) every second.
- Uses pygame.time.delay() or pygame.time.Clock() for timing.
- The color should change smoothly in an infinite loop until the window is closed.

4. Text Blinking with Random Colors

Develop a PyGame script that:

- Displays blinking text “Learning PyGame!” at the center of the screen.
- Randomly changes the text color from a list of at least 6 colors.
- The blinking effect should happen every 0.8 seconds.

5. Adding a Rectangular Border around Text

Create a program that:

- Displays a text “Game On” centered on the screen.
- Draws a black rectangular border around the text with a margin of 10 pixels.
- The text color should change every second while the border remains static.
- The border should automatically resize if the text string is changed.

6. Complete “Title Screen” Simulation

Develop a **game title screen** using PyGame that:

- Displays the text “My First Game” at the center.
- Includes a border, a background color, and a smaller subtitle “Press Enter to Start.”
- The title should blink between two colors every second.
- When the user presses **Enter**, the screen clears and displays “Game Starting…” for 2 seconds before quitting.
- Use structured functions (like draw_title(), draw_subtitle(), and main()).